

DSA STUDY BLUEPRINT SERIES

117. Dijkstras Algorithm Single Source Shortest Path Greedy

Single Source Shortest Path for Weighted Graphs (Dijkstra)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Dijkstra:** Resolves shortest path using a min-heap priority queue.
- Does not support negative edge weights.

Memory & Execution Blueprint

Active node 1 distance = 4 • relaxes adjacent node 2
to distance $4 + 2 = 6$.

C++ Implementation Reference

Standard code layout and syntax

```
vector dijkstra(int V, vector>>& adj, int src) {  
    vector dist(V, 1e9); dist[src] = 0;  
    priority_queue<vector>, greater>> pq;  
    pq.push({0, src});  
    while (!pq.empty()) {  
        int u = pq.top().second; pq.pop();  
        for (auto edge : adj[u]) {  
            int v = edge.first, w = edge.second;  
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }  
        }  
    }  
    return dist;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Priority Queue	$O((V + E) \log V)$ time, $O(V)$ space

Key Practices & Gotchas

- **Important:** Using a min-heap ensures we greedily resolve paths from the closest active node.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.